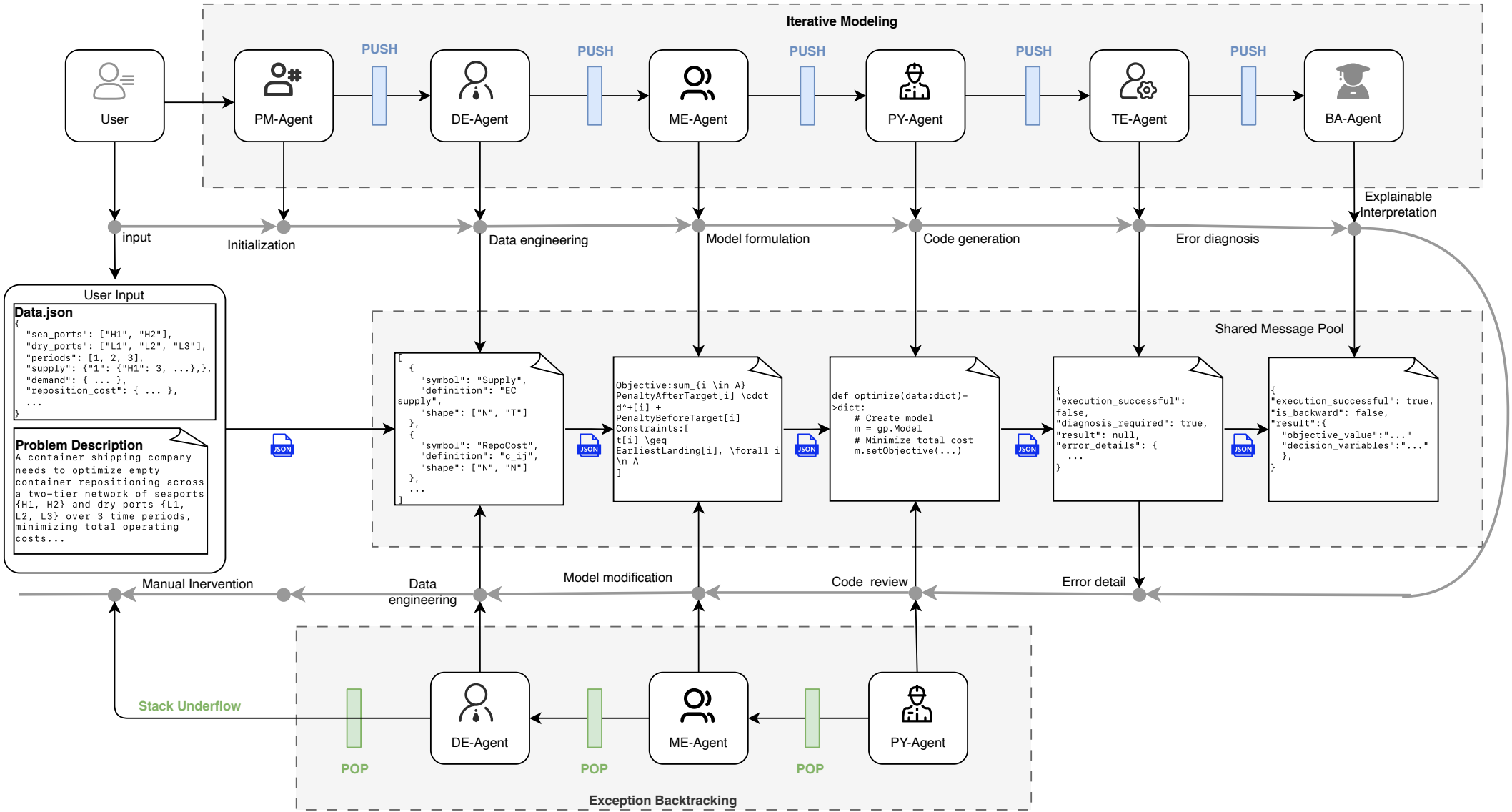
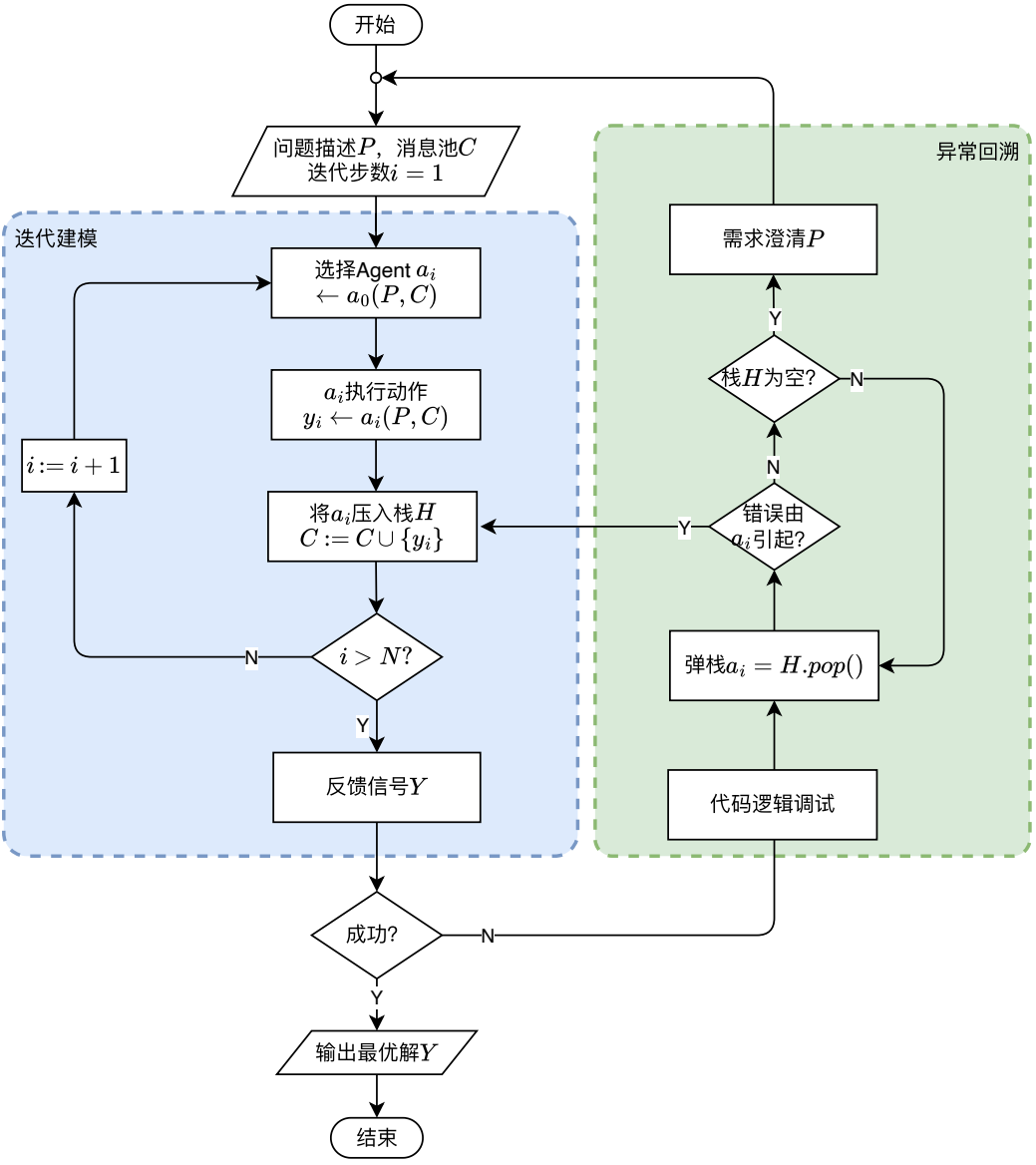
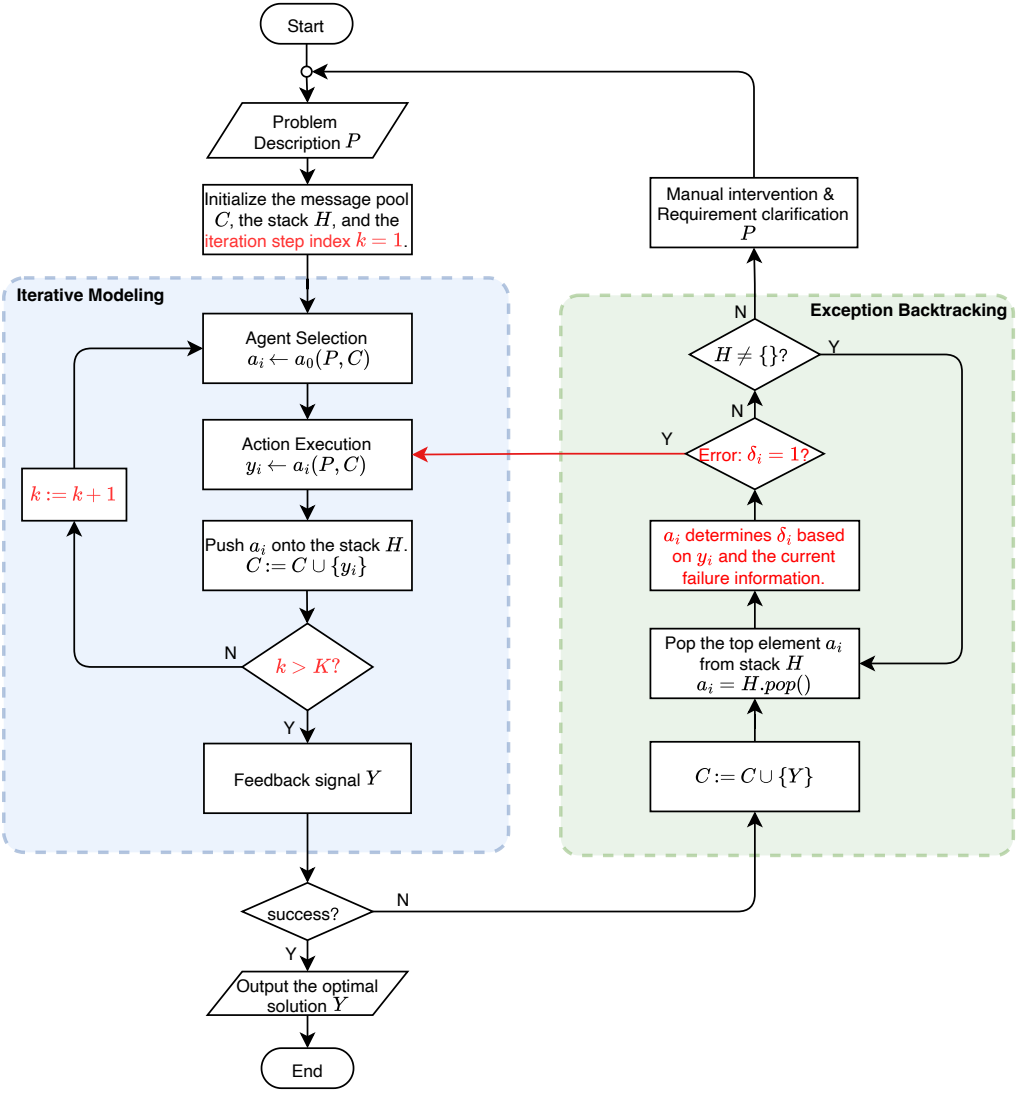

Iterative Modeling
User
PM-Agent
DE-Agent
ME-Agent
PY-Agent
TE-Agent
BA-Agent
PUSH
PUSH
PUSH
PUSH
PUSH
input
Initialization
Data engineering
Model formulation
Code generation
Eror diagnosis
Explainable Interpretation
User Input
Data.json
{
"sea_ports": ["H1", "H2"],
"dry_ports": ["L1", "L2", "L3"],
"periods": [1, 2, 3],
"supply": {"1": {"H1": 3, ...},},
"demand": { ... },
"reposition_cost": { ... },
...
}
Problem Description
A container shipping company needs to optimize empty container repositioning across a two-tier network of seaports {H1, H2} and dry ports {L1, L2, L3} over 3 time periods, minimizing total operating costs...
Shared Message Pool
[
{
"symbol": "Supply",
"definition": "EC supply",
"shape": ["N", "T"]
},
{
"symbol": "RepoCost",
"definition": "c_ij",
"shape": ["N", "N"]
},
...
]
Objective:sum_{i \in A} PenaltyAfterTarget[i] \cdot d^+[i] + PenaltyBeforeTarget[i]
Constraints:[
t[i] \geq EarliestLanding[i], \forall i \n A
]
def optimize(data:dict)->dict:
# Create model
m = gp.Model
# Minimize total cost
m.setObjective(...)
{
"execution_successful": false,
"diagnosis_required": true,
"result": null,
"error_details": {
...
}
{
"execution_successful": true,
"is_backward": false,
"result":{
"objective_value":"..."
"decision_variables":"..."
},
}
JSON
Manual Inervention
Data engineering
Model modification
Code review
Error detail
Stack Underflow
DE-Agent
ME-Agent
PY-Agent
POP
POP
POP
Exception Backtracking


开始
问题描述$P$，消息池$C$
迭代步数$i = 1$
迭代建模
选择Agent $a_i$
$\leftarrow a_0(P, C)$
$a_i$执行动作
$y_i \leftarrow a_i(P, C)$
将$a_i$压入栈$H$
$C := C \cup \{y_i\}$
$i > N$?
N
$i := i + 1$
Y
反馈信号$Y$
成功?
N
Y
输出最优解$Y$
结束
异常回溯
代码逻辑调试
弹栈$a_i = H.pop()$
错误由
$a_i$引起?
Y
N
栈$H$为空?
N
Y
需求澄清$P$


Start
Problem Description $P$
Initialize the message pool $C$, the stack $H$, and the iteration step index $k = 1$.
Iterative Modeling
Agent Selection $a_i \leftarrow a_0(P, C)$
Action Execution $y_i \leftarrow a_i(P, C)$
Push $a_i$ onto the stack $H$. $C := C \cup \{y_i\}$
$k > K$?
N
Y
$k := k + 1$
Feedback signal $Y$
success?
N
Y
Output the optimal solution $Y$
End
Manual intervention & Requirement clarification $P$
Exception Backtracking
$H \neq \{\}$?
N
Y
Error: $\delta_i = 1$?
N
Y
$a_i$ determines $\delta_i$ based on $y_i$ and the current failure information.
Pop the top element $a_i$ from stack $H$ $a_i = H.pop()$
$C := C \cup \{Y\}$